CQRS Pattern: An Example

Let's consider a simple application that manages user profiles. In this application, you can create a new user (command), update their email (command), and view user details (query).

1. Command Model (Write Model)

1. **Command Class**: Defines an action to be performed. For instance, `CreateUserCommand` with fields like `userId`, `name`, and `email`.
2. **Command Handler**: Processes the commands. It would take `CreateUserCommand`, perform business logic, and persist the changes.
3. **Domain Model**: Represents the state of the data (e.g., a `User` class).
4. **Repository**: Handles data persistence.

2. Query Model (Read Model)

1. **DTO (Data Transfer Object)**: Represents the data we want to read, like `UserProfileDTO.`
2. **Query Class**: Defines a query, for instance, `GetUserProfileQuery` with a `userId`.
3. **Query Handler**: Handles the query and returns the requested data, often using a read-optimized storage.

3. Infrastructure

1. **Command Bus**: Routes commands to their respective handlers.
2. **Query Bus**: Routes queries to their respective handlers.
3. **Event Store**: In a more advanced CQRS setup, this would store events resulting from commands.

Here is the sample code for the aforementioned components:

Command Model (Write Model)

1. Command Class

```
public class CreateUserCommand {
    private String userId;
    private String name;
    private String email;

    // Constructor, Getters, and Setters
}

public class UpdateUserEmailCommand {
    private String userId;
    private String newEmail;

    // Constructor, Getters, and Setters
}
```

2. Command Handler

```
public class CreateUserCommandHandler {
    private UserRepository userRepository;

    public CreateUserCommandHandler(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public void handle(CreateUserCommand command) {
        User user = new User(command.getUserId(), command.getName(),
command.getEmail());
        userRepository.save(user);
    }
}

public class UpdateUserEmailCommandHandler {
    private UserRepository userRepository;

    public UpdateUserEmailCommandHandler(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public void handle(UpdateUserEmailCommand command) {
        if (!isValidEmail(command.getNewEmail())) {
            throw new IllegalArgumentException("Invalid email format");
        }
        User user = userRepository.findById(command.getUserId());
        if (user != null) {
            user.setEmail(command.getNewEmail());
            userRepository.save(user);
        }
    }

    private boolean isValidEmail(String email) {
        // Simple email validation logic
        return email != null && email.contains("@");
    }
}
```

3. Domain Model

```
public class User {
    private String id;
    private String name;
    private String email;

    // Constructor, Getters, and Setters
}
```

4. Repository

```
public interface UserRepository {
    void save(User user);
    User findById(String id);
}

public class UserRepositoryImpl implements UserRepository {
    // Implementation of save and findById methods
}
```

Query Model (Read Model)

1. DTO (Data Transfer Object)

```
public class UserProfileDTO {
    private String id;
    private String name;
    private String email;

    // Constructor, Getters, and Setters
}
```

2. Query Class

```
public class GetUserProfileQuery {
    private String userId;

    // Constructor, Getters, and Setters
}

// Read Email Query
public class ReadUserEmailQuery {
    private String userId;

    // Constructor, Getters, and Setters
}
```

3. Query Handler

```
public class GetUserProfileQueryHandler {
    private UserRepository userRepository;

    public GetUserProfileQueryHandler(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public UserProfileDTO handle(GetUserProfileQuery query) {
        User user = userRepository.findById(query.getUserId());
        return new UserProfileDTO(user.getId(), user.getName(), user.getEmail());
    }
}

// Read Email Query Handler
public class ReadUserEmailQueryHandler {
    private UserRepository userRepository;

    public ReadUserEmailQueryHandler(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public String handle(ReadUserEmailQuery query) {
        User user = userRepository.findById(query.getUserId());
        return user != null ? user.getEmail() : null;
    }
}
```

Infrastructure

1. Command Bus

```
public interface CommandBus {
    void dispatch(CreateUserCommand command);
}

public class SimpleCommandBus implements CommandBus {
    private CreateUserCommandHandler handler;

    public SimpleCommandBus(CreateUserCommandHandler handler) {
        this.handler = handler;
    }

    @Override
    public void dispatch(CreateUserCommand command) {
        handler.handle(command);
    }
}
```

2. Query Bus

```
public interface QueryBus {
    UserProfileDTO dispatch(GetUserProfileQuery query);
}

public class SimpleQueryBus implements QueryBus {
    private GetUserProfileQueryHandler handler;

    public SimpleQueryBus(GetUserProfileQueryHandler handler) {
        this.handler = handler;
    }

    @Override
    public UserProfileDTO dispatch(GetUserProfileQuery query) {
        return handler.handle(query);
    }
}
```

3. Event Store (Simplified Example)

```
public interface EventStore {
    void storeEvent(UserEvent event);
}

public class SimpleEventStore implements EventStore {
    private List<UserEvent> events = new ArrayList<>();

    @Override
    public void storeEvent(UserEvent event) {
        events.add(event);
    }
}
```

Putting It All Together

Java

```
public class CQRSApplication {
    public static void main(String[] args) {
        UserRepository userRepository = new UserRepositoryImpl();
        CreateUserCommandHandler createHandler = new CreateUserCommandHandler(userReposi
        UpdateUserEmailCommandHandler updateEmailHandler = new UpdateUserEmailCommandHar
        GetUserProfileQueryHandler queryHandler = new GetUserProfileQueryHandler(userRep
        ReadUserEmailQueryHandler readEmailHandler = new ReadUserEmailQueryHandler(userR

        CommandBus commandBus = new SimpleCommandBus(createHandler, updateEmailHandler);
        QueryBus queryBus = new SimpleQueryBus(queryHandler, readEmailHandler);

        // Create User
        CreateUserCommand createCommand = new CreateUserCommand("1", "Alice", "alice@exa
        commandBus.dispatch(createCommand);

        // Update Email
        UpdateUserEmailCommand updateEmailCommand = new UpdateUserEmailCommand("1", "ali
        commandBus.dispatch(updateEmailCommand);

        // Read Updated Email
        ReadUserEmailQuery readEmailQuery = new ReadUserEmailQuery("1");
```

This example includes a basic implementation of a command bus, query bus, and a simplified event store. In a full-fledged application, an event store would be used for event sourcing, which is often paired with CQRS. The event store captures changes as a series of events, which can be replayed to rebuild the state of an entity.

← Previous
The Inner Workings Of The CQRS Pattern

Next →
Issues, Special Considerations, And Performance Implications

☐ Mark as Completed